

Rapport individuel

Périmètre conteneurisation et application Go

Auteur

Karim Haddadi

Client

Korlea

Mastère DevOps M1, SUP DE VINCI Paris La Défense

Promotion M1DOA 2025-2026

1. Pourquoi ce document existe

Le DAT groupe couvre déjà l'architecture, les choix techniques et les KPIs. Ce rapport ne les répète pas. Il raconte mon vécu sur le périmètre application Go et conteneurisation : les incidents que j'ai débogués, les compromis que je subis, les apprentissages qui ont changé mes réflexes, et les chantiers que je prioriserais après la démo. La référence reste le DAT, en particulier §4.3 (Conteneurisation) et §4.6 (Application métier) que j'ai portés.

2. Perspectives d'évolution

Trois priorités post-démo qui touchent directement mon périmètre, plus quelques pistes que je garde pour le M2.

1. Backup et HA Postgres. Pourquoi pour moi : ma table `audit_log` et mon `CronJob audit-purge` reposent intégralement sur une instance CNPG unique, sur `local-path` non répliqué, sans aucun backup. Si la base disparaît, la traçabilité RGPD disparaît avec. Dépendance : Tom (stockage répliqué côté flake), Nicolas (observabilité de la base). Mise en œuvre : étendre `database.yaml` avec `backup.barmanObjectStore` vers un stockage S3-compatible et passer à `instances: 3` en réplication synchrone, puis valider que mes probes tiennent le failover.

2. Remettre OIDC en route. Pourquoi pour moi : mon code `internal/auth/auth.go` supporte les trois rôles (`anonymous`, `viewer`, `operator`) avec mapping group claim, mais en démo toute requête passe en `operator`, mon audit log perd de sa valeur. Dépendance : Nicolas (rotation client secret Dex via Vault), décision client (GitHub OIDC, LDAP ou SAML). Mise en œuvre : rotater le secret, décommenter les variables `OIDC_*` du Deployment api, brancher un connecteur Dex.

3. Dependabot pour calmer Trivy. Le scan Trivy est strict sur GHCR, zéro CVE HIGH ou CRITICAL accepté, et sans bump automatique en amont il bloque parfois une PR pour une CVE qu'un simple coup de Dependabot sur une dépendance Go ou la base distroless aurait corrigée. Ça dépend surtout de la matrice CI d'Amine et des `required status checks` ; un `dependabot.yml` hebdomadaire groupé minor/patch suffit à le mettre en place.

Pistes M2, hors DAT. *SBOM et cosign* : générer un SBOM via `syft` à chaque build et l'attacher comme attestation cosign à côté de l'image rendrait la supply chain vérifiable côté cluster, deux actions GitHub à ajouter. *HPA worker sur queue depth* : la configuration `External Metrics` basée sur `jobs_queue_depth` via le Prometheus adapter est déjà en commentaire dans le manifest HPA, il manque le déploiement de l'adapter côté plateforme. *Debug eBPF sans shell* : `pixie`, `parca`, ou `bpftrace` en sidecar privilégié, dans la continuité du choix « pas de `kubect1 exec` » adopté pendant le projet.

3. Limites techniques et incidents

3.1 L'incident qui m'a fait construire ma checklist GitOps

Le problème le plus chronophage que j'ai rencontré sur mon périmètre tient en une phrase : la CI est verte, l'image est sur GHCR, mais le cluster n'a pas la bonne version. Symptômes typiques : `ImagePullBackOff` au déploiement d'un nouveau Pod, ou ancien Pod qui tourne encore alors que je viens de merger une PR sur `main`.

Le piège, c'est que ce n'est ni « que » mon Dockerfile, ni « que » le pipeline build d'Amine. C'est le digest dans `kustomization.yaml`, le write-back de l'`argocd-image-updater`, et la confusion entre le tag `main` (mutable, suivi par l'`updater`) et le digest immuable (réellement déployé). On

cherchait au mauvais endroit. Mon réflexe initial était de regarder le Dockerfile ou les logs du build. Le bon réflexe, c'est de regarder Git.

La checklist qui sort de cet incident, on la suit maintenant à chaque fois qu'une image semble ne pas avoir pris :

1. la CI de build est verte sur le commit attendu ;
2. le digest dans *kustomization.yaml* a été mis à jour par l'argocd-image-updater (commit visible dans l'historique du dépôt infra) ;
3. l'Application ArgoCD est en **Synced** plus **Healthy** ;
4. si non, **kubectl describe pod** plus logs ciblés, en distinguant trois familles de problèmes : image (pull, registry, digest), base (Postgres pas prêt, ExternalSecret en attente), ou probe (readinessProbe qui refuse parce qu'un **pgxpool.Ping** échoue).

Règle d'or qui en découle : le rollback applicatif passe par un revert Git du digest dans *kustomization.yaml*, pas par **kubectl rollout undo**. Le **rollout undo** ramène un état que GitOps ne connaît pas, et ArgoCD le remet à jour à la prochaine sync, donc on tourne en rond.

Cette checklist marche bien en flux normal. Le risque résiduel reste un merge applicatif plus une PR infra le même jour sans se parler : si Tom ou Amine change quelque chose côté ArgoCD ou pipeline en parallèle d'un nouveau digest, le diagnostic devient un dialogue plutôt qu'un check solo. On gère, mais c'est là que la coordination compte.

3.2 Le passage à distroless et la fin de **kubectl exec**

Le choix distroless a été tranché tôt dans le projet, après comparatif. Image runtime **gcr.io/distroless/static-debian12:nonroot**, binaire Go statique avec **CGO_ENABLED=0**, taille finale entre 25 et 35 MB par image. Sur le papier, c'est le bon choix : surface d'attaque réduite, conformité PSS Restricted alignée par défaut, **nonroot** UID 65532 imposé.

En pratique, le coût n'est pas dans le build. Il est dans le debug. Quand un Pod api ne répond pas comme attendu, le réflexe naturel est **kubectl exec -it ... -- sh**. Sur distroless, il n'y a pas de shell. Pas de **sh**, pas de **bash**, pas de **curl**, rien. Frustrant au début.

Le déclic est venu quand j'ai accepté de remplacer ce réflexe par un autre triplet : les logs JSON structurés produits par **slog**, les métriques Prometheus exposées par mon package **metrics**, et les traces Tempo via le tracer OTel. Tout est en place côté observabilité grâce au travail de Nicolas, qui a poussé la stack avant que j'en aie réellement besoin. Le debug ne consiste plus à entrer dans le conteneur, il consiste à suivre une trace : **trace_id** dans le log, exemplar dans le bucket d'histogramme Prometheus, navigation directe vers Tempo, lecture du span échoué, retour aux logs structurés du Pod incriminé filtrés par **trace_id**. L'enchaînement reste sous la minute pour un incident type.

Le compromis assumé : sur un incident vraiment exotique (un binaire qui se bloque avant d'émettre une trace, une corruption locale du filesystem éphémère), je n'ai pas d'option côté Pod. Il faut redéployer une image debug temporaire ou activer un sidecar privilégié. Sur la durée du projet, ce cas ne s'est pas présenté.

3.3 Les compromis que je subis ou que j'ai assumés

Cinq compromis pèsent sur mon périmètre, trois que j'assume avec l'équipe et deux que je subis.

Côté assumés, l'OIDC coupé pour la démo fait que mon RBAC à trois rôles ne tourne pas vraiment : toutes les actions passent par le même **operator** fictif et l'audit log perd un peu de sens. Le **DEMO_CHAOS_ENABLED=1** hard-codé dans le Deployment api laisse Nicolas déclencher les endpoints chaos pendant la démo, utile là et dangereux en prod, mais le retirer ne coûte qu'une

PR à un commit une fois la démo Korlea passée. Et Trivy bloque parfois une PR pour une CVE non corrigée upstream : quand le fix amont n'existe pas, il ne reste qu'un waiver à durée ou un patch local, ni l'un ni l'autre rapide, c'est le prix d'un Trivy strict.

Côté subis, Postgres en single-instance sur `local-path` se voit dans mes probes : pendant un rolling update de la base ou l'arrivée d'un Pod api sur un nœud sans cache image, ma `readinessProbe` peut basculer `Not Ready` quelques secondes. Le `startupProbe` budget 30 s que j'ai posé tient ce délai ; en multi-instances il disparaîtrait, et ça je ne peux pas le corriger seul. Testcontainers, lui, demande Docker en local : natif en CI `ubuntu-latest`, absent du devShell Nix, donc je passe par le démon Docker host ou `lima/colima`. Ce n'est documenté nulle part, ça se redécouvre membre par membre.

4. Analyse personnelle

4.1 Ce que j'ai vraiment appris

Distroless. Avant le projet, debug = shell dans le conteneur. Après, debug = `trace_id` qu'on suit dans les logs, les métriques et Tempo. Pas juste une nouvelle compétence : un déplacement de réflexe. Une fois qu'on accepte qu'il n'y a pas de shell, on s'interdit les solutions qui en dépendraient, et on investit côté observabilité.

Cache Docker par composant. Sur le papier, un détail buildx. En pratique, ça transforme la cadence : rebuild incrémental autour d'une minute au lieu de quatre ou cinq, on push une PR sans réfléchir au coût. C'est aussi le sujet où j'ai compris que la CI d'Amine n'est pas une couche au-dessus de la mienne mais une vraie collaboration : on règle ensemble le scope par composant et l'ordre des étapes Dockerfile.

Testcontainers contre les mocks. J'ai démarré le projet avec l'idée qu'un mock Postgres serait plus rapide. J'ai changé d'avis vite : un Postgres `16-alpine` éphémère par test détecte des bugs que les mocks cachent (typage `INET`, migrations idempotentes, sémantique exacte de `LISTEN/NOTIFY`). Plus lent en wallclock, meilleur payoff sur la qualité.

4.2 Ce qui marche, ce qui me coûte

Ma force, c'est de tenir la chaîne complète `code` → `image` → `manifest` → `cluster` et de débbugger quand ça casse à l'interface CI/K3s. Quand un Pod `projet-etude-app-demo-*` ne se comporte pas comme attendu, je sais ouvrir la bonne page sans demander.

Ce qui me coûte, c'est dès qu'on sort de mes manifests. HA Postgres, backups, stockage distribué, Vault prod, Tailscale avancé : je comprends, je ne pilote pas seul. Tom et Nicolas portent ces sujets, je suis utilisateur de leurs choix.

4.3 Compétences acquises

Côté technique : Go en contraintes prod, Docker multi-stage distroless avec posture sécurité, Kubernetes côté manifests applicatifs (Deployment, Service, HPA, PDB, NetworkPolicy, Cron-Job), instrumentation OTel via *otelpgx*, et surtout le GitOps des images par digest plutôt que par tag mutable.

Côté transverse, j'ai appris à travailler en vraie dépendance. Mon Dockerfile ne marche pas sans Amine, mes Deployments ne se synchronisent pas sans le bootstrap ArgoCD de Tom, mes Pods n'ont pas leurs secrets sans External-Secrets que Nicolas pilote. Nommer ce qui dépend de qui, demander quand il faut, écrire des PRs qui ne court-circuitent pas les autres.

5. Ce que j'emporte

Quatre choses que je ferais autrement et qui synthétisent l'expérience du projet.

OIDC dès le départ, quitte à pointer un Dex local minimal, pour forcer la rotation du client secret et le mapping `groups` upstream en début de projet plutôt qu'à la démo. La checklist image/digest dès le premier déploiement ArgoCD, plutôt qu'après l'incident raconté en §3.1. Un paragraphe README pour Docker en local des tests d'intégration, ou une recette `just test-local`. Dependabot dès le premier commit Go, pour que Trivy soit moins frustrant en aval.

Ce que je retiens au-delà du projet : la chaîne CI/K3s est un vrai sujet de coordination, pas une simple succession technique, et l'observabilité (logs structurés + métriques + traces) remplace mieux qu'on ne croit l'accès shell à un conteneur. Pour la suite, j'irai chercher SBOM/cosign, HPA sur métrique métier et debug eBPF, qui prolongent naturellement les réflexes adoptés ici.

6. Annexe, documentation utilisateur : ajouter un nouveau composant conteneurisé

Cette annexe décrit la procédure qu'on suit pour ajouter un composant au dépôt application, du code jusqu'au cluster. Elle vise un dev qui arrive sur le projet et veut livrer une nouvelle brique sans casser la chaîne. Cinq étapes, dans l'ordre.

1. Le Dockerfile. On ajoute une cible au build multi-stage existant. Le stage de build part de `golang:1.25-alpine`, compile en statique avec `CGO_ENABLED=0`, et l'image finale est `gcr.io/distroless/static-debian12:nonroot`. On copie le binaire, on expose le port si le composant sert du HTTP (l'api est sur 8080, le worker sur 8081, l'audit-purge n'expose rien), et c'est tout. Pas de shell, pas de paquet en plus.
2. La matrice CI. On ajoute le composant à la matrice de `.github/workflows/build.yml` pour qu'il passe les mêmes étapes que les autres : `golangci-lint`, test race, scan Trivy bloquant sur HIGH et CRITICAL avant le push GHCR, puis build et push avec son propre scope de cache (`type=gha, mode=max`). Sans son entrée dans la matrice, le composant n'est ni testé ni scanné.
3. Les manifests K8s. On crée le Deployment dans `kubernetes/apps/projet-etude-app-demo/`, le Service s'il reçoit du trafic, et les ressources qui vont avec : HPA, PDB, et surtout une NetworkPolicy en deny-by-default qui déclare explicitement les egress autorisés. SecurityContext `nonroot` (UID 65532), `readOnlyRootFilesystem`, et les trois probes startup, readiness, liveness. Les egress nominaux, à adapter au besoin réel : DNS vers CoreDNS dans `kube-system` (UDP et TCP 53), Postgres CNPG en TCP 5432 vers le bon Service et namespace, la chaîne OTLP vers le collecteur puis Tempo en gRPC 4317 ou HTTP 4318, HTTPS 443 vers l'issuer Dex (JWKS et metadata) si l'OIDC est réactivé, HTTPS 443 vers des FQDN explicites pour les webhooks métier (jamais l'internet ouvert), et HTTPS 443 vers `kubernetes.default` seulement si le Pod doit parler à l'API Kubernetes. À ne pas mettre par défaut sur le même Pod : le pull d'images (c'est le kubelet), le scrape Prometheus (c'est de l'ingress), et l'accès aux secrets, qui passe par ExternalSecrets et pas par Vault côté application selon notre modèle.
4. La kustomization et l'image-updater. On référence la nouvelle image dans le `kustomization.yaml` du namespace, et on l'ajoute à l'annotation `image-list` de l'argocd-image-updater. Le write-back se fait par Git, par digest, sur le tag `:main`. Le déploiement réel passe par le digest immuable écrit dans Git, pas par le tag mutable. Si on oublie cette étape, l'image se build mais le cluster ne la voit jamais.

5. La validation observabilité. On ajoute un **ServiceMonitor** pour le scrape Prometheus, puis on vérifie que les métriques sortent, que les logs remontent vers Loki et que les traces arrivent à Tempo. Tant que le composant n'apparaît pas dans Grafana, il n'est pas vraiment livré : sur distroless, c'est la seule fenêtre qu'on a dessus.